

PLATEFORME DE VEILLE ISO/IEC 17025

Arborescence complète et documentée

Rôle de chaque fichier, organisation du code et explication des mécanismes essentiels

219 fichiers inventoriés - dépendances node_modules, dist, coverage, output et tmp exclus

Version documentaire du 20 septembre 2026

Sommaire

Sommaire	2
1. Comment lire cette documentation	5
2. Vue complète de l’arborescence	6
2. Vue complète de l’arborescence - suite	7
2. Vue complète de l’arborescence - suite	8
2. Vue complète de l’arborescence - suite	9
2. Vue complète de l’arborescence - suite	10
3. Répartition des fichiers	11
4. Fiches fichier par fichier	12
Backend / .gitignore	12
Backend / .prettierrc	12
Backend / actions	12
Backend / app.module.ts	12
Backend / audit	12
Backend / auth	13
Backend / collection	14
Backend / connectors	14
Backend / dashboard	15
Backend / database	15
Backend / favorites	16
Backend / health	16
Backend / main.ts	16
Backend / nest-cli.json	16
Backend / notifications	17
Backend / oxlint.json	17
Backend / package-lock.json	17
Backend / package.json	17
Backend / permissions	17
Backend / qualification	18
Backend / reports	18
Backend / roles	18
Backend / saved-views	19
Backend / search	19
Backend / sources	19
Backend / storage	20

Backend / subscriptions 20

Backend / taxonomy 21

Backend / test 22

Backend / tsconfig.build.json 22

Backend / tsconfig.build.tsbuildinfo 22

Backend / tsconfig.json 22

Backend / users 22

Backend / validation 23

Backend / vitest.config.e2e.mts 23

Backend / vitest.config.e2e.ts 24

Backend / vitest.config.mts 24

Backend / watch-items 24

Frontend / .gitignore 24

Frontend / App.vue 25

Frontend / assets 25

Frontend / i18n 25

Frontend / index.html 25

Frontend / layouts 25

Frontend / main.ts 25

Frontend / package-lock.json 26

Frontend / package.json 26

Frontend / public 26

Frontend / router 26

Frontend / services 26

Frontend / stores 27

Frontend / style.css 27

Frontend / tsconfig.app.json 28

Frontend / tsconfig.json 28

Frontend / tsconfig.node.json 28

Frontend / views 28

Frontend / vite.config.ts 29

Frontend / vitest.config.ts 29

Racine du projet 29

5. Explication des mécanismes de code essentiels 30

5.1 main.ts : sécuriser toutes les requêtes 30

5.2 AppModule : composition de l'application 30

5.3 Contrôleur → service → repository 30

5.4 Guards et rôles 30

5.5 Transactions 30

5.6 Événements 30

5.7 Recherche plein texte	30
5.8 Déduplication	30
5.9 Store d'authentification	31
5.10 Chargement différé des pages	31
6. Ordre conseillé pour comprendre le projet	32

1. Comment lire cette documentation

L'inventaire couvre tous les fichiers propres au projet présents au moment de la génération. Les dossiers de dépendances et de compilation sont exclus car ils sont recréés automatiquement. Les fichiers .env ne sont volontairement pas reproduits : ils contiennent des secrets et sont ignorés par Git.

Suffixe / dossier	Signification
*.controller.ts	Routes HTTP et autorisations
*.service.ts	Règles métier
*.module.ts	Assemblage NestJS
*.entity.ts	Table et relations PostgreSQL
dto/	Validation des entrées
*.vue	Page ou composant utilisateur
frontend/src/services/	Appels vers l'API
*.spec.ts	Tests automatisés

Chaque fiche donne le chemin, la catégorie, le rôle précis et, lorsque le fichier en contient, les classes, fonctions, routes ou rôles importants.

2. Vue complète de l'arborescence

```
plateforme-veille/
├── backend/
│   └── src/
│       ├── actions/
│       │   ├── constants/
│       │   │   └── action-status.constants.ts
│       │   ├── dto/
│       │   │   ├── create-action.dto.ts
│       │   │   └── update-action.dto.ts
│       │   ├── entities/
│       │   │   └── follow-up-action.entity.ts
│       │   ├── actions.controller.ts
│       │   ├── actions.module.ts
│       │   └── actions.service.ts
│       ├── audit/
│       │   ├── entities/
│       │   │   └── audit-log.entity.ts
│       │   ├── audit.controller.ts
│       │   ├── audit.module.ts
│       │   └── audit.service.ts
│       ├── auth/
│       │   ├── decorators/
│       │   │   └── roles.decorator.ts
│       │   ├── dto/
│       │   │   └── login.dto.ts
│       │   ├── guards/
│       │   │   ├── jwt-auth.guard.ts
│       │   │   └── roles.guard.ts
│       │   ├── interfaces/
│       │   │   └── jwt-payload.interface.ts
│       │   ├── auth.controller.spec.ts
│       │   ├── auth.controller.ts
│       │   ├── auth.module.ts
│       │   ├── auth.service.spec.ts
│       │   ├── auth.service.ts
│       │   └── jwt.strategy.ts
│       ├── collection/
│       │   ├── entities/
│       │   │   └── collection-run.entity.ts
│       │   ├── collection-runs.controller.ts
│       │   ├── collection-scheduler.service.ts
│       │   ├── collection.controller.ts
│       │   ├── collection.module.ts
│       │   ├── collection.service.ts
│       │   └── normalization.service.ts
│       ├── connectors/
│       │   ├── dto/
│       │   │   └── create-connector.dto.ts
│       │   ├── entities/
│       │   │   └── connector.entity.ts
│       │   ├── implementations/
│       │   │   ├── crossref.connector.ts
│       │   │   ├── manual.connector.ts
│       │   │   └── rss.connector.ts
│       │   ├── interfaces/
│       │   │   └── connector.interface.ts
│       │   ├── utils/
│       │   │   └── safe-http-client.ts
│       │   ├── connectors.controller.ts
│       │   ├── connectors.module.ts
│       │   └── connectors.service.ts
│       ├── dashboard/
│       │   ├── dashboard.controller.ts
│       │   ├── dashboard.module.ts
│       │   └── dashboard.service.ts
│       └── database/
│           ├── migrations/
│           │   ├── 1789403962682-AddModule4Qualification.ts
│           │   ├── 1789576535050-AddModule5ValidationTracking.ts
│           │   ├── 1789730353043-AddSearchFavoritesSavedViews.ts
│           │   ├── 1789743631190-AddModule6SubscriptionsNotificationsReports.ts
│           │   └── 1789770445842-AddAuditLogs.ts
```

2. Vue complète de l'arborescence - suite

```
|
|
|
|  ├── data-source.ts
|  ├── database-seed.service.ts
|  └── database.module.ts
|
|  ├── favorites/
|  |   ├── entities/
|  |   |   └── favorite.entity.ts
|  |   ├── favorites.controller.ts
|  |   ├── favorites.module.ts
|  |   └── favorites.service.ts
|  |
|  ├── health/
|  |   ├── health.controller.ts
|  |   ├── health.module.ts
|  |   └── health.service.ts
|  |
|  ├── notifications/
|  |   ├── entities/
|  |   |   └── notification.entity.ts
|  |   ├── events/
|  |   |   └── watch-item-published.event.ts
|  |   ├── notifications.controller.ts
|  |   ├── notifications.module.ts
|  |   └── notifications.service.ts
|  |
|  ├── permissions/
|  |   ├── entities/
|  |   |   └── permission.entity.ts
|  |   ├── permissions.module.ts
|  |   ├── permissions.service.spec.ts
|  |   └── permissions.service.ts
|  |
|  ├── qualification/
|  |   ├── dto/
|  |   |   └── qualify-watch-item.dto.ts
|  |   ├── qualification.controller.ts
|  |   ├── qualification.module.ts
|  |   └── qualification.service.ts
|  |
|  ├── reports/
|  |   ├── dto/
|  |   |   └── generate-report.dto.ts
|  |   ├── entities/
|  |   |   └── report.entity.ts
|  |   ├── reports.controller.ts
|  |   ├── reports.module.ts
|  |   └── reports.service.ts
|  |
|  ├── roles/
|  |   ├── entities/
|  |   |   └── role.entity.ts
|  |   ├── roles.controller.ts
|  |   ├── roles.module.ts
|  |   ├── roles.service.spec.ts
|  |   └── roles.service.ts
|  |
|  ├── saved-views/
|  |   ├── dto/
|  |   |   └── create-saved-view.dto.ts
|  |   ├── entities/
|  |   |   └── saved-view.entity.ts
|  |   ├── saved-views.controller.ts
|  |   ├── saved-views.module.ts
|  |   └── saved-views.service.ts
|  |
|  ├── search/
|  |   ├── dto/
|  |   |   └── search-watch-items.dto.ts
|  |   ├── search.controller.ts
|  |   ├── search.module.ts
|  |   └── search.service.ts
|  |
|  ├── sources/
|  |   ├── dto/
|  |   |   ├── create-source.dto.ts
|  |   |   ├── update-source-status.dto.ts
|  |   |   └── update-source.dto.ts
|  |   ├── entities/
|  |   |   └── source.entity.ts
|  |   ├── sources.controller.ts
|  |   ├── sources.module.ts
|  |   └── sources.service.ts
```

2. Vue complète de l'arborescence - suite

```

├── subscriptions/
│   ├── dto/
│   │   └── create-subscription.dto.ts
│   ├── entities/
│   │   └── subscription.entity.ts
│   ├── subscriptions.controller.ts
│   ├── subscriptions.module.ts
│   └── subscriptions.service.ts
├── taxonomy/
│   ├── dto/
│   │   └── taxonomy.dto.ts
│   ├── entities/
│   │   ├── domain.entity.ts
│   │   ├── keyword-synonym.entity.ts
│   │   ├── keyword.entity.ts
│   │   ├── laboratory.entity.ts
│   │   └── topic.entity.ts
│   ├── taxonomy.controller.ts
│   ├── taxonomy.module.ts
│   └── taxonomy.service.ts
├── users/
│   ├── dto/
│   │   ├── create-user.dto.ts
│   │   ├── update-user-roles.dto.ts
│   │   └── update-user-status.dto.ts
│   ├── entities/
│   │   └── user.entity.ts
│   ├── users.controller.spec.ts
│   ├── users.controller.ts
│   ├── users.module.ts
│   ├── users.service.spec.ts
│   └── users.service.ts
├── validation/
│   ├── dto/
│   │   ├── review-watch-item.dto.ts
│   │   └── workflow-comment.dto.ts
│   ├── entities/
│   │   └── review.entity.ts
│   ├── validation.controller.ts
│   ├── validation.module.ts
│   └── validation.service.ts
├── watch-items/
│   ├── constants/
│   │   └── watch-status.constants.ts
│   ├── entities/
│   │   ├── watch-item-keyword.entity.ts
│   │   ├── watch-item-topic.entity.ts
│   │   ├── watch-item.entity.ts
│   │   └── watch-version.entity.ts
│   ├── interfaces/
│   │   └── normalized-watch-item.interface.ts
│   ├── deduplication.service.ts
│   ├── watch-items.controller.ts
│   ├── watch-items.module.ts
│   └── watch-items.service.ts
├── app.module.ts
├── main.ts
├── storage/
│   └── reports/
│       ├── rapport-veille-hebdomadaire-2026-06-17-au-2026-09-18-1789768748060.pdf
│       ├── rapport-veille-mensuel-2026-02-26-au-2026-09-18-1789769107451.csv
│       ├── rapport-veille-mensuel-2026-05-14-au-2026-09-18-1789769092577.xlsx
│       ├── report-1789746421720.pdf
│       ├── report-1789746729065.xlsx
│       ├── report-1789746802166.csv
│       └── report-1789750751633.pdf
├── test/
│   ├── app.e2e-spec.ts
│   ├── dashboard.cjs
│   ├── taxonomy.cjs
│   └── users-roles.cjs
└── .gitignore

```

2. Vue complète de l'arborescence - suite

```
├── .prettierrc
├── nest-cli.json
├── oxlint.json
├── package-lock.json
├── package.json
├── tsconfig.build.json
├── tsconfig.build.tsbuildinfo
├── tsconfig.json
├── vitest.config.e2e.mts
├── vitest.config.e2e.ts
├── vitest.config.mts
├── frontend/
│   ├── public/
│   │   └── favicon.svg
│   └── src/
│       ├── assets/
│       │   └── logos/
│       │   └── logo-veille-microscope.png
│       ├── i18n/
│       │   ├── errors.spec.ts
│       │   ├── errors.ts
│       │   ├── labels.spec.ts
│       │   ├── labels.ts
│       │   └── primevue-fr.ts
│       ├── layouts/
│       │   └── AppLayout.vue
│       ├── router/
│       │   └── index.ts
│       ├── services/
│       │   ├── taxonomy/
│       │   │   └── taxonomy.service.ts
│       │   ├── actions.service.ts
│       │   ├── api.ts
│       │   ├── audit.service.ts
│       │   ├── connector-config.ts
│       │   ├── dashboard.service.ts
│       │   ├── favorites.service.ts
│       │   ├── health.service.ts
│       │   ├── notifications.service.ts
│       │   ├── qualification.service.ts
│       │   ├── reports.service.ts
│       │   ├── saved-views.service.ts
│       │   ├── search.service.ts
│       │   ├── sources.service.ts
│       │   ├── subscriptions.service.ts
│       │   ├── taxonomy.service.ts
│       │   ├── users.service.ts
│       │   ├── validation.service.ts
│       │   └── watch-items.service.ts
│       ├── stores/
│       │   └── auth.ts
│       ├── views/
│       │   ├── ActionsView.vue
│       │   ├── AuditLogsView.vue
│       │   ├── DashboardView.vue
│       │   ├── LoginView.vue
│       │   ├── NotificationsView.vue
│       │   ├── ReportsView.vue
│       │   ├── SourcesView.vue
│       │   ├── SubscriptionsView.vue
│       │   ├── SystemHealthView.vue
│       │   ├── TaxonomyView.vue
│       │   ├── UsersView.vue
│       │   ├── WatchItemDetailView.vue
│       │   ├── WatchItemQualificationView.vue
│       │   └── WatchItemsView.vue
│       ├── App.vue
│       ├── main.ts
│       └── style.css
├── .gitignore
├── index.html
└── package-lock.json
```

2. Vue complète de l'arborescence - suite

```
| | package.json
| | tsconfig.app.json
| | tsconfig.json
| | tsconfig.node.json
| | vite.config.ts
| | vitest.config.ts
| | .gitignore
| | package-lock.json
| | package.json
| | README.md
```

3. Répartition des fichiers

Zone	Nombre	Responsabilité
backend	160	API, métier, persistance, sécurité et traitements
frontend	55	Interface, navigation, état et accès API
racine	4	Documentation et configuration globale

4. Fiches fichier par fichier

Backend / .gitignore

[backend/.gitignore](#)

Fichier projet - Participe au fonctionnement général du projet.

Taille actuelle : 686 octets

Backend / .prettierrc

[backend/.prettierrc](#)

Fichier projet - Participe au fonctionnement général du projet.

Taille actuelle : 56 octets

Backend / actions

[backend/src/actions/actions.controller.ts](#)

Contrôleur API - Expose les routes HTTP, applique les Guards et transmet les données au service métier. Routes principales : POST /api/v1/watch-items/:id/actions, GET /api/v1/watch-items/:id/actions, GET /api/v1/actions, PATCH /api/v1/actions/:id. Éléments déclarés : ActionsController. Rôles cités : ADMIN, REFERENT_LABORATOIRE, RESPONSABLE_VEILLE.

Taille actuelle : 1 730 octets

[backend/src/actions/actions.module.ts](#)

Module NestJS - Assemble les contrôleurs, services, repositories et dépendances du module « actions ». Éléments déclarés : ActionsModule.

Taille actuelle : 806 octets

[backend/src/actions/actions.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « actions » et dialogue avec les repositories ou services associés. Éléments déclarés : ActionsService. Rôles cités : ADMIN, REFERENT_LABORATOIRE, RESPONSABLE_VEILLE.

Taille actuelle : 4 777 octets

[backend/src/actions/constants/action-status.constants.ts](#)

Constantes métier - Centralise des statuts ou valeurs autorisées pour éviter les chaînes dispersées. Éléments déclarés : ActionStatus, ACTION_STATUS.

Taille actuelle : 219 octets

[backend/src/actions/dto/create-action.dto.ts](#)

DTO de validation - Décrit et valide les données acceptées par une route avant leur arrivée dans le service. Éléments déclarés : CreateActionDto.

Taille actuelle : 551 octets

[backend/src/actions/dto/update-action.dto.ts](#)

DTO de validation - Décrit et valide les données acceptées par une route avant leur arrivée dans le service. Éléments déclarés : UpdateActionDto.

Taille actuelle : 393 octets

[backend/src/actions/entities/follow-up-action.entity.ts](#)

Entité TypeORM - Mappe la table PostgreSQL « actions », ses colonnes et ses relations. Éléments déclarés : FollowUpAction.

Taille actuelle : 1 162 octets

Backend / app.module.ts

[backend/src/app.module.ts](#)

Module NestJS - Assemble les contrôleurs, services, repositories et dépendances du module « app ». Éléments déclarés : AppModule.

Taille actuelle : 5 378 octets

Backend / audit

[backend/src/audit/audit.controller.ts](#)

Contrôleur API - Expose les routes HTTP, applique les Guards et transmet les données au service métier. Routes principales : GET /api/v1/audit. Éléments déclarés : AuditController. Rôles cités : ADMIN, RESPONSABLE_VEILLE.

Taille actuelle : 539 octets

[backend/src/audit/audit.module.ts](#)

Module NestJS - Assemble les contrôleurs, services, repositories et dépendances du module « audit ». Éléments déclarés : AuditModule.

Taille actuelle : 618 octets

[backend/src/audit/audit.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « audit » et dialogue avec les repositories ou services associés. Éléments déclarés : AuditService.

Taille actuelle : 1 229 octets

[backend/src/audit/entities/audit-log.entity.ts](#)

Entité TypeORM - Mappe la table PostgreSQL « audit_logs », ses colonnes et ses relations. Éléments déclarés : AuditLog.

Taille actuelle : 995 octets

Backend / auth

[backend/src/auth/auth.controller.spec.ts](#)

Test automatisé - Vérifie le comportement du composant ou service associé avec Vitest. Rôles cités : ADMIN.

Taille actuelle : 1 357 octets

[backend/src/auth/auth.controller.ts](#)

Contrôleur API - Expose les routes HTTP, applique les Guards et transmet les données au service métier. Routes principales : POST /api/v1/auth/login, POST /api/v1/auth/logout, GET /api/v1/auth/me. Éléments déclarés : AuthController.

Taille actuelle : 1 348 octets

[backend/src/auth/auth.module.ts](#)

Module NestJS - Assemble les contrôleurs, services, repositories et dépendances du module « auth ». Éléments déclarés : AuthModule.

Taille actuelle : 900 octets

[backend/src/auth/auth.service.spec.ts](#)

Test automatisé - Vérifie le comportement du composant ou service associé avec Vitest. Rôles cités : ADMIN.

Taille actuelle : 2 090 octets

[backend/src/auth/auth.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « auth » et dialogue avec les repositories ou services associés. Éléments déclarés : AuthService.

Taille actuelle : 1 570 octets

[backend/src/auth/decorators/roles.decorator.ts](#)

Décorateur NestJS - Ajoute une métadonnée réutilisable, notamment la liste des rôles autorisés. Éléments déclarés : ROLES_KEY, Roles.

Taille actuelle : 164 octets

[backend/src/auth/dto/login.dto.ts](#)

DTO de validation - Décrit et valide les données acceptées par une route avant leur arrivée dans le service. Éléments déclarés : LoginDto.

Taille actuelle : 222 octets

[backend/src/auth/guards/jwt-auth.guard.ts](#)

Guard de sécurité - Autorise ou refuse une requête selon l'authentification ou les rôles. Éléments déclarés : JwtAuthGuard.

Taille actuelle : 163 octets

[backend/src/auth/guards/roles.guard.ts](#)

Guard de sécurité - Autorise ou refuse une requête selon l'authentification ou les rôles. Éléments déclarés : RolesGuard.

Taille actuelle : 892 octets

[backend/src/auth/interfaces/jwt-payload.interface.ts](#)

Contrat TypeScript - Définit la forme des données échangées sans produire de code à l'exécution. Éléments déclarés : JwtPayload.

Taille actuelle : 88 octets

[backend/src/auth/jwt.strategy.ts](#)

Fichier projet - Participe au fonctionnement général du projet. Éléments déclarés : JwtStrategy.

Taille actuelle : 1 396 octets

Backend / collection

[backend/src/collection/collection-runs.controller.ts](#)

Contrôleur API - Expose les routes HTTP, applique les Guards et transmet les données au service métier. Routes principales : GET /api/v1/collection-runs. Éléments déclarés : CollectionRunsController. Rôles cités : ADMIN, RESPONSABLE_VEILLE.

Taille actuelle : 627 octets

[backend/src/collection/collection-scheduler.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « collection » et dialogue avec les repositories ou services associés. Éléments déclarés : CollectionSchedulerService.

Taille actuelle : 2 023 octets

[backend/src/collection/collection.controller.ts](#)

Contrôleur API - Expose les routes HTTP, applique les Guards et transmet les données au service métier. Routes principales : POST /api/v1/connectors/:id/run, POST /api/v1/connectors/:id/retry. Éléments déclarés : CollectionController. Rôles cités : ADMIN, RESPONSABLE_VEILLE.

Taille actuelle : 900 octets

[backend/src/collection/collection.module.ts](#)

Module NestJS - Assemble les contrôleurs, services, repositories et dépendances du module « collection ». Éléments déclarés : CollectionModule.

Taille actuelle : 1 187 octets

[backend/src/collection/collection.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « collection » et dialogue avec les repositories ou services associés. Éléments déclarés : CollectionService.

Taille actuelle : 5 324 octets

[backend/src/collection/entities/collection-run.entity.ts](#)

Entité TypeORM - Mappe la table PostgreSQL « collection_runs », ses colonnes et ses relations. Éléments déclarés : CollectionRun.

Taille actuelle : 1 241 octets

[backend/src/collection/normalization.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « collection » et dialogue avec les repositories ou services associés. Éléments déclarés : NormalizationService.

Taille actuelle : 2 671 octets

Backend / connectors

[backend/src/connectors/connectors.controller.ts](#)

Contrôleur API - Expose les routes HTTP, applique les Guards et transmet les données au service métier. Routes principales : GET /api/v1/connectors, POST /api/v1/connectors, POST /api/v1/connectors/:id/test. Éléments déclarés : ConnectorsController. Rôles cités : ADMIN, RESPONSABLE_VEILLE.

Taille actuelle : 1 004 octets

[backend/src/connectors/connectors.module.ts](#)

Module NestJS - Assemble les contrôleurs, services, repositories et dépendances du module « connectors ». Éléments déclarés : ConnectorsModule.

Taille actuelle : 665 octets

[backend/src/connectors/connectors.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « connectors » et dialogue avec les repositories ou services associés. Éléments déclarés : ConnectorsService.

Taille actuelle : 4 427 octets

[backend/src/connectors/dto/create-connector.dto.ts](#)

DTO de validation - Décrit et valide les données acceptées par une route avant leur arrivée dans le service. Éléments déclarés : CreateConnectorDto.

Taille actuelle : 279 octets

[backend/src/connectors/entities/connector.entity.ts](#)

Entité TypeORM - Mappe la table PostgreSQL « connectors », ses colonnes et ses relations. Éléments déclarés : Connector.

Taille actuelle : 892 octets

[backend/src/connectors/implementations/crossref.connector.ts](#)

Implémentation de connecteur - Traduit un fournisseur concret vers le contrat commun BaseConnector. Éléments déclarés : CrossrefConnector.

Taille actuelle : 4 823 octets

[backend/src/connectors/implementations/manual.connector.ts](#)

Implémentation de connecteur - Traduit un fournisseur concret vers le contrat commun BaseConnector. Éléments déclarés : ManualConnector.

Taille actuelle : 418 octets

[backend/src/connectors/implementations/rss.connector.ts](#)

Implémentation de connecteur - Traduit un fournisseur concret vers le contrat commun BaseConnector. Éléments déclarés : RssConnector.

Taille actuelle : 3 430 octets

[backend/src/connectors/interfaces/connector.interface.ts](#)

Contrat TypeScript - Définit la forme des données échangées sans produire de code à l'exécution. Éléments déclarés : ConnectorTestResult, ExternalItem, CollectionResult, BaseConnector.

Taille actuelle : 488 octets

[backend/src/connectors/utils/safe-http-client.ts](#)

Utilitaire sécurisé - Fournit une fonction technique partagée et encapsule ses contrôles de sécurité. Éléments déclarés : SafeHttpResponse, safeGet.

Taille actuelle : 6 043 octets

Backend / dashboard

[backend/src/dashboard/dashboard.controller.ts](#)

Contrôleur API - Expose les routes HTTP, applique les Guards et transmet les données au service métier. Routes principales : GET /api/v1/dashboard, GET /api/v1/dashboard/details. Éléments déclarés : DashboardController.

Taille actuelle : 641 octets

[backend/src/dashboard/dashboard.module.ts](#)

Module NestJS - Assemble les contrôleurs, services, repositories et dépendances du module « dashboard ». Éléments déclarés : DashboardModule.

Taille actuelle : 915 octets

[backend/src/dashboard/dashboard.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « dashboard » et dialogue avec les repositories ou services associés. Éléments déclarés : DashboardService. Rôles cités : ADMIN, OPERATEUR_VEILLE, REFERENT_LABORATOIRE, RESPONSABLE_VEILLE.

Taille actuelle : 9 358 octets

Backend / database

[backend/src/database/data-source.ts](#)

Source de données TypeORM - Configuration utilisée par la CLI pour générer, afficher et exécuter les migrations.

Taille actuelle : 454 octets

[backend/src/database/database-seed.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « database » et dialogue avec les repositories ou services associés. Éléments déclarés : DatabaseSeedService. Rôles cités : ADMIN, LECTEUR, OPERATEUR_VEILLE, REFERENT_LABORATOIRE, RESPONSABLE_VEILLE.

Taille actuelle : 2 656 octets

[backend/src/database/database.module.ts](#)

Module NestJS - Assemble les contrôleurs, services, repositories et dépendances du module « database ». Éléments déclarés : DatabaseModule.

Taille actuelle : 441 octets

[backend/src/database/migrations/1789403962682-AddModule4Qualification.ts](#)

Migration TypeORM - Décrit une évolution versionnée du schéma PostgreSQL et son annulation dans down(). Éléments déclarés : AddModule4Qualification1789403962682.

Taille actuelle : 8 291 octets

[backend/src/database/migrations/1789576535050-AddModule5ValidationTracking.ts](#)

Migration TypeORM - Décrit une évolution versionnée du schéma PostgreSQL et son annulation dans down(). Éléments déclarés : AddModule5ValidationTracking1789576535050.

Taille actuelle : 2 567 octets

[backend/src/database/migrations/1789730353043-AddSearchFavoritesSavedViews.ts](#)

Migration TypeORM - Décrit une évolution versionnée du schéma PostgreSQL et son annulation dans down(). Éléments déclarés : AddSearchFavoritesSavedViews1789730353043.

Taille actuelle : 2 243 octets

[backend/src/database/migrations/1789743631190-AddModule6SubscriptionsNotificationsReports.ts](#)

Migration TypeORM - Décrit une évolution versionnée du schéma PostgreSQL et son annulation dans down(). Éléments déclarés : AddModule6SubscriptionsNotificationsReports1789743631190.

Taille actuelle : 4 593 octets

[backend/src/database/migrations/1789770445842-AddAuditLogs.ts](#)

Migration TypeORM - Décrit une évolution versionnée du schéma PostgreSQL et son annulation dans down(). Éléments déclarés : AddAuditLogs1789770445842.

Taille actuelle : 1 077 octets

Backend / favorites

[backend/src/favorites/entities/favorite.entity.ts](#)

Entité TypeORM - Mappe la table PostgreSQL « favorites », ses colonnes et ses relations. Éléments déclarés : Favorite.

Taille actuelle : 716 octets

[backend/src/favorites/favorites.controller.ts](#)

Contrôleur API - Expose les routes HTTP, applique les Guards et transmet les données au service métier. Routes principales : GET /api/v1/favorites, POST /api/v1/favorites/:itemId, DELETE /api/v1/favorites/:itemId. Éléments déclarés : FavoritesController.

Taille actuelle : 943 octets

[backend/src/favorites/favorites.module.ts](#)

Module NestJS - Assemble les contrôleurs, services, repositories et dépendances du module « favorites ». Éléments déclarés : FavoritesModule.

Taille actuelle : 699 octets

[backend/src/favorites/favorites.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « favorites » et dialogue avec les repositories ou services associés. Éléments déclarés : FavoritesService.

Taille actuelle : 1 594 octets

Backend / health

[backend/src/health/health.controller.ts](#)

Contrôleur API - Expose les routes HTTP, applique les Guards et transmet les données au service métier. Routes principales : GET /api/v1/health. Éléments déclarés : HealthController. Rôles cités : ADMIN, RESPONSABLE_VEILLE.

Taille actuelle : 545 octets

[backend/src/health/health.module.ts](#)

Module NestJS - Assemble les contrôleurs, services, repositories et dépendances du module « health ». Éléments déclarés : HealthModule.

Taille actuelle : 543 octets

[backend/src/health/health.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « health » et dialogue avec les repositories ou services associés. Éléments déclarés : HealthService.

Taille actuelle : 976 octets

Backend / main.ts

[backend/src/main.ts](#)

Point d'entrée backend - Crée NestJS, active Helmet, cookies, limites de requête, CORS, validation globale et écoute le port configuré.

Taille actuelle : 1 402 octets

Backend / nest-cli.json

[backend/nest-cli.json](#)

Configuration NestJS - Configure le dossier source et le compilateur du backend NestJS.

Taille actuelle : 179 octets

Backend / notifications

[backend/src/notifications/entities/notification.entity.ts](#)

Entité TypeORM - Mappe la table PostgreSQL « notifications », ses colonnes et ses relations. Éléments déclarés : Notification.

Taille actuelle : 1 113 octets

[backend/src/notifications/events/watch-item-published.event.ts](#)

Événement métier - Transporte les informations nécessaires à un traitement découplé après publication. Éléments déclarés :

WatchItemPublishedEvent.

Taille actuelle : 98 octets

[backend/src/notifications/notifications.controller.ts](#)

Contrôleur API - Expose les routes HTTP, applique les Guards et transmet les données au service métier. Routes principales : GET /api/v1/notifications, PATCH /api/v1/notifications/:id/read. Éléments déclarés : NotificationsController.

Taille actuelle : 763 octets

[backend/src/notifications/notifications.module.ts](#)

Module NestJS - Assemble les contrôleurs, services, repositories et dépendances du module « notifications ». Éléments déclarés : NotificationsModule.

Taille actuelle : 808 octets

[backend/src/notifications/notifications.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « notifications » et dialogue avec les repositories ou services associés. Éléments déclarés : NotificationsService.

Taille actuelle : 2 701 octets

Backend / oxlint.json

[backend/oxlint.json](#)

Qualité - Configure l'analyse statique Oxlint du backend.

Taille actuelle : 269 octets

Backend / package-lock.json

[backend/package-lock.json](#)

Fichier généré npm - Fige les versions exactes et l'arbre des dépendances pour des installations reproductibles.

Taille actuelle : 264 637 octets

Backend / package.json

[backend/package.json](#)

Configuration npm - Déclare les scripts, dépendances et métadonnées du paquet concerné.

Taille actuelle : 2 488 octets

Backend / permissions

[backend/src/permissions/entities/permission.entity.ts](#)

Entité TypeORM - Mappe la table PostgreSQL « permissions », ses colonnes et ses relations. Éléments déclarés : Permission.

Taille actuelle : 479 octets

[backend/src/permissions/permissions.module.ts](#)

Module NestJS - Assemble les contrôleurs, services, repositories et dépendances du module « permissions ». Éléments déclarés : PermissionsModule.

Taille actuelle : 403 octets

[backend/src/permissions/permissions.service.spec.ts](#)

Test automatisé - Vérifie le comportement du composant ou service associé avec Vitest.

Taille actuelle : 738 octets

[backend/src/permissions/permissions.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « permissions » et dialogue avec les repositories ou services associés. Éléments déclarés : PermissionsService.

Taille actuelle : 448 octets

Backend / qualification

[backend/src/qualification/dto/qualify-watch-item.dto.ts](#)

DTO de validation - Décrit et valide les données acceptées par une route avant leur arrivée dans le service. Éléments déclarés : QualifyWatchItemDto.

Taille actuelle : 743 octets

[backend/src/qualification/qualification.controller.ts](#)

Contrôleur API - Expose les routes HTTP, applique les Guards et transmet les données au service métier. Routes principales : GET /api/v1/watch-items/:id/qualification, PATCH /api/v1/watch-items/:id/qualification. Éléments déclarés : QualificationController. Rôles cités : ADMIN, OPERATEUR_VEILLE, RESPONSABLE_VEILLE.

Taille actuelle : 1 045 octets

[backend/src/qualification/qualification.module.ts](#)

Module NestJS - Assemble les contrôleurs, services, repositories et dépendances du module « qualification ». Éléments déclarés : QualificationModule.

Taille actuelle : 1 182 octets

[backend/src/qualification/qualification.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « qualification » et dialogue avec les repositories ou services associés. Éléments déclarés : QualificationService.

Taille actuelle : 4 930 octets

Backend / reports

[backend/src/reports/dto/generate-report.dto.ts](#)

DTO de validation - Décrit et valide les données acceptées par une route avant leur arrivée dans le service. Éléments déclarés : GenerateReportDto.

Taille actuelle : 281 octets

[backend/src/reports/entities/report.entity.ts](#)

Entité TypeORM - Mappe la table PostgreSQL « reports », ses colonnes et ses relations. Éléments déclarés : Report.

Taille actuelle : 894 octets

[backend/src/reports/reports.controller.ts](#)

Contrôleur API - Expose les routes HTTP, applique les Guards et transmet les données au service métier. Routes principales : POST /api/v1/reports, GET /api/v1/reports, GET /api/v1/reports/:id/download. Éléments déclarés : ReportsController. Rôles cités : ADMIN, RESPONSABLE_VEILLE.

Taille actuelle : 1 546 octets

[backend/src/reports/reports.module.ts](#)

Module NestJS - Assemble les contrôleurs, services, repositories et dépendances du module « reports ». Éléments déclarés : ReportsModule.

Taille actuelle : 680 octets

[backend/src/reports/reports.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « reports » et dialogue avec les repositories ou services associés. Éléments déclarés : ReportsService.

Taille actuelle : 9 040 octets

Backend / roles

[backend/src/roles/entities/role.entity.ts](#)

Entité TypeORM - Mappe la table PostgreSQL « roles », ses colonnes et ses relations. Éléments déclarés : Role.

Taille actuelle : 745 octets

[backend/src/roles/roles.controller.ts](#)

Contrôleur API - Expose les routes HTTP, applique les Guards et transmet les données au service métier. Routes principales : GET /api/v1/roles. Éléments déclarés : RolesController. Rôles cités : ADMIN.

Taille actuelle : 535 octets

[backend/src/roles/roles.module.ts](#)

Module NestJS - Assemble les contrôleurs, services, repositories et dépendances du module « roles ». Éléments déclarés : RolesModule.

Taille actuelle : 558 octets

[backend/src/roles/roles.service.spec.ts](#)

Test automatisé - Vérifie le comportement du composant ou service associé avec Vitest.

Taille actuelle : 685 octets

[backend/src/roles/roles.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « roles » et dialogue avec les repositories ou services associés. Éléments déclarés : RolesService.

Taille actuelle : 642 octets

Backend / saved-views

[backend/src/saved-views/dto/create-saved-view.dto.ts](#)

DTO de validation - Décrit et valide les données acceptées par une route avant leur arrivée dans le service. Éléments déclarés : CreateSavedViewDto.

Taille actuelle : 221 octets

[backend/src/saved-views/entities/saved-view.entity.ts](#)

Entité TypeORM - Mappe la table PostgreSQL « saved_views », ses colonnes et ses relations. Éléments déclarés : SavedView.

Taille actuelle : 591 octets

[backend/src/saved-views/saved-views.controller.ts](#)

Contrôleur API - Expose les routes HTTP, applique les Guards et transmet les données au service métier. Routes principales : GET /api/v1/saved-views, POST /api/v1/saved-views, DELETE /api/v1/saved-views/:id. Éléments déclarés : SavedViewsController.

Taille actuelle : 962 octets

[backend/src/saved-views/saved-views.module.ts](#)

Module NestJS - Assemble les contrôleurs, services, repositories et dépendances du module « saved-views ». Éléments déclarés : SavedViewsModule.

Taille actuelle : 629 octets

[backend/src/saved-views/saved-views.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « saved-views » et dialogue avec les repositories ou services associés. Éléments déclarés : SavedViewsService.

Taille actuelle : 1 287 octets

Backend / search

[backend/src/search/dto/search-watch-items.dto.ts](#)

DTO de validation - Décrit et valide les données acceptées par une route avant leur arrivée dans le service. Éléments déclarés : SearchWatchItemsDto.

Taille actuelle : 1 072 octets

[backend/src/search/search.controller.ts](#)

Contrôleur API - Expose les routes HTTP, applique les Guards et transmet les données au service métier. Routes principales : GET /api/v1/search/watch-items. Éléments déclarés : SearchController.

Taille actuelle : 707 octets

[backend/src/search/search.module.ts](#)

Module NestJS - Assemble les contrôleurs, services, repositories et dépendances du module « search ». Éléments déclarés : SearchModule.

Taille actuelle : 551 octets

[backend/src/search/search.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « search » et dialogue avec les repositories ou services associés. Éléments déclarés : SearchService. Rôles cités : ADMIN, OPERATEUR_VEILLE, REFERENT_LABORATOIRE, RESPONSABLE_VEILLE.

Taille actuelle : 2 701 octets

Backend / sources

[backend/src/sources/dto/create-source.dto.ts](#)

DTO de validation - Décrit et valide les données acceptées par une route avant leur arrivée dans le service. Éléments déclarés : CreateSourceDto.

Taille actuelle : 660 octets

[backend/src/sources/dto/update-source-status.dto.ts](#)

DTO de validation - Décrit et valide les données acceptées par une route avant leur arrivée dans le service. Éléments déclarés : UpdateSourceStatusDto.

Taille actuelle : 145 octets

[backend/src/sources/dto/update-source.dto.ts](#)

DTO de validation - Décrit et valide les données acceptées par une route avant leur arrivée dans le service. Éléments déclarés : UpdateSourceDto.

Taille actuelle : 178 octets

[backend/src/sources/entities/source.entity.ts](#)

Entité TypeORM - Mappe la table PostgreSQL « sources », ses colonnes et ses relations. Éléments déclarés : Source.

Taille actuelle : 1 227 octets

[backend/src/sources/sources.controller.ts](#)

Contrôleur API - Expose les routes HTTP, applique les Guards et transmet les données au service métier. Routes principales : GET /api/v1/sources, GET /api/v1/sources/:id, POST /api/v1/sources, PATCH /api/v1/sources/:id, PATCH /api/v1/sources/:id/status. Éléments déclarés : SourcesController. Rôles cités : ADMIN, OPERATEUR_VEILLE, RESPONSABLE_VEILLE.

Taille actuelle : 1 770 octets

[backend/src/sources/sources.module.ts](#)

Module NestJS - Assemble les contrôleurs, services, repositories et dépendances du module « sources ». Éléments déclarés : SourcesModule.

Taille actuelle : 647 octets

[backend/src/sources/sources.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « sources » et dialogue avec les repositories ou services associés. Éléments déclarés : SourcesService.

Taille actuelle : 3 039 octets

Backend / storage

[backend/storage/reports/rapport-veille-hebdomadaire-2026-06-17-au-2026-09-18-1789768748060.pdf](#)

Rapport généré - Fichier de rapport déjà produit et conservé par le backend pour téléchargement.

Taille actuelle : 4 888 octets

[backend/storage/reports/rapport-veille-mensuel-2026-02-26-au-2026-09-18-1789769107451.csv](#)

Rapport généré - Fichier de rapport déjà produit et conservé par le backend pour téléchargement.

Taille actuelle : 1 177 octets

[backend/storage/reports/rapport-veille-mensuel-2026-05-14-au-2026-09-18-1789769092577.xlsx](#)

Rapport généré - Fichier de rapport déjà produit et conservé par le backend pour téléchargement.

Taille actuelle : 8 253 octets

[backend/storage/reports/report-1789746421720.pdf](#)

Rapport généré - Fichier de rapport déjà produit et conservé par le backend pour téléchargement.

Taille actuelle : 1 502 octets

[backend/storage/reports/report-1789746729065.xlsx](#)

Rapport généré - Fichier de rapport déjà produit et conservé par le backend pour téléchargement.

Taille actuelle : 6 835 octets

[backend/storage/reports/report-1789746802166.csv](#)

Rapport généré - Fichier de rapport déjà produit et conservé par le backend pour téléchargement.

Taille actuelle : 32 octets

[backend/storage/reports/report-1789750751633.pdf](#)

Rapport généré - Fichier de rapport déjà produit et conservé par le backend pour téléchargement.

Taille actuelle : 1 826 octets

Backend / subscriptions

[backend/src/subscriptions/dto/create-subscription.dto.ts](#)

DTO de validation - Décrit et valide les données acceptées par une route avant leur arrivée dans le service. Éléments déclarés : CreateSubscriptionDto.

Taille actuelle : 405 octets

[backend/src/subscriptions/entities/subscription.entity.ts](#)

Entité TypeORM - Mappe la table PostgreSQL « subscriptions », ses colonnes et ses relations. Éléments déclarés : Subscription.

Taille actuelle : 1 471 octets

[backend/src/subscriptions/subscriptions.controller.ts](#)

Contrôleur API - Expose les routes HTTP, applique les Guards et transmet les données au service métier. Routes principales : GET /api/v1/subscriptions/options, GET /api/v1/subscriptions, POST /api/v1/subscriptions, DELETE /api/v1/subscriptions/:id. Éléments déclarés : SubscriptionsController.

Taille actuelle : 1 053 octets

[backend/src/subscriptions/subscriptions.module.ts](#)

Module NestJS - Assemble les contrôleurs, services, repositories et dépendances du module « subscriptions ». Éléments déclarés : SubscriptionsModule.

Taille actuelle : 1 023 octets

[backend/src/subscriptions/subscriptions.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « subscriptions » et dialogue avec les repositories ou services associés. Éléments déclarés : SubscriptionsService.

Taille actuelle : 3 460 octets

Backend / taxonomy

[backend/src/taxonomy/dto/taxonomy.dto.ts](#)

DTO de validation - Décrit et valide les données acceptées par une route avant leur arrivée dans le service. Éléments déclarés : CreateTopicDto, UpdateTopicDto, CreateNamedTermDto, UpdateNamedTermDto, CreateKeywordDto, UpdateKeywordDto, CreateSynonymDto, UpdateSynonymDto.

Taille actuelle : 1 718 octets

[backend/src/taxonomy/entities/domain.entity.ts](#)

Entité TypeORM - Mappe la table PostgreSQL « domains », ses colonnes et ses relations. Éléments déclarés : Domain.

Taille actuelle : 562 octets

[backend/src/taxonomy/entities/keyword-synonym.entity.ts](#)

Entité TypeORM - Mappe la table PostgreSQL « keyword_synonyms », ses colonnes et ses relations. Éléments déclarés : KeywordSynonym.

Taille actuelle : 507 octets

[backend/src/taxonomy/entities/keyword.entity.ts](#)

Entité TypeORM - Mappe la table PostgreSQL « keywords », ses colonnes et ses relations. Éléments déclarés : Keyword.

Taille actuelle : 611 octets

[backend/src/taxonomy/entities/laboratory.entity.ts](#)

Entité TypeORM - Mappe la table PostgreSQL « laboratories », ses colonnes et ses relations. Éléments déclarés : Laboratory.

Taille actuelle : 576 octets

[backend/src/taxonomy/entities/topic.entity.ts](#)

Entité TypeORM - Mappe la table PostgreSQL « topics », ses colonnes et ses relations. Éléments déclarés : Topic.

Taille actuelle : 577 octets

[backend/src/taxonomy/taxonomy.controller.ts](#)

Contrôleur API - Expose les routes HTTP, applique les Guards et transmet les données au service métier. Routes principales : GET /api/v1/taxonomy/topics, GET /api/v1/taxonomy/topics/:id, POST /api/v1/taxonomy/topics, PATCH /api/v1/taxonomy/topics/:id, DELETE /api/v1/taxonomy/topics/:id, GET /api/v1/taxonomy/domains, GET /api/v1/taxonomy/domains/:id, POST /api/v1/taxonomy/domains. Éléments déclarés : TaxonomyController. Rôles cités : ADMIN.

Taille actuelle : 4 113 octets

[backend/src/taxonomy/taxonomy.module.ts](#)

Module NestJS - Assemble les contrôleurs, services, repositories et dépendances du module « taxonomy ». Éléments déclarés : TaxonomyModule.

Taille actuelle : 838 octets

[backend/src/taxonomy/taxonomy.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « taxonomy » et dialogue avec les repositories ou services associés. Éléments déclarés : TaxonomyService, TaxonomyKind.

Taille actuelle : 4 107 octets

Backend / test

[backend/test/app.e2e-spec.ts](#)

Fichier projet - Participe au fonctionnement général du projet. Rôles cités : ADMIN.

Taille actuelle : 2 505 octets

[backend/test/dashboard.cjs](#)

Script de test manuel - Script CommonJS d'aide pour interroger rapidement une fonctionnalité du backend. Rôles cités : ADMIN, LECTEUR, OPERATEUR_VEILLE, REFERENT_LABORATOIRE, RESPONSABLE_VEILLE.

Taille actuelle : 2 695 octets

[backend/test/taxonomy.cjs](#)

Script de test manuel - Script CommonJS d'aide pour interroger rapidement une fonctionnalité du backend. Rôles cités : ADMIN, LECTEUR.

Taille actuelle : 4 498 octets

[backend/test/users-roles.cjs](#)

Script de test manuel - Script CommonJS d'aide pour interroger rapidement une fonctionnalité du backend. Rôles cités : ADMIN, LECTEUR, RESPONSABLE_VEILLE.

Taille actuelle : 2 443 octets

Backend / tsconfig.build.json

[backend/tsconfig.build.json](#)

Configuration TypeScript - Définit les règles de compilation TypeScript pour ce périmètre.

Taille actuelle : 178 octets

Backend / tsconfig.build.tsbuildinfo

[backend/tsconfig.build.tsbuildinfo](#)

Configuration TypeScript - Définit les règles de compilation TypeScript pour ce périmètre.

Taille actuelle : 289 185 octets

Backend / tsconfig.json

[backend/tsconfig.json](#)

Configuration TypeScript - Définit les règles de compilation TypeScript pour ce périmètre.

Taille actuelle : 600 octets

Backend / users

[backend/src/users/dto/create-user.dto.ts](#)

DTO de validation - Décrit et valide les données acceptées par une route avant leur arrivée dans le service. Éléments déclarés : CreateUserDto.

Taille actuelle : 752 octets

[backend/src/users/dto/update-user-roles.dto.ts](#)

DTO de validation - Décrit et valide les données acceptées par une route avant leur arrivée dans le service. Éléments déclarés : UpdateUserRolesDto.

Taille actuelle : 374 octets

[backend/src/users/dto/update-user-status.dto.ts](#)

DTO de validation - Décrit et valide les données acceptées par une route avant leur arrivée dans le service. Éléments déclarés : UpdateUserStatusDto.

Taille actuelle : 143 octets

[backend/src/users/entities/user.entity.ts](#)

Entité TypeORM - Mappe la table PostgreSQL « users », ses colonnes et ses relations. Éléments déclarés : User.

Taille actuelle : 1 189 octets

[backend/src/users/users.controller.spec.ts](#)

Test automatisé - Vérifie le comportement du composant ou service associé avec Vitest.

Taille actuelle : 862 octets

[backend/src/users/users.controller.ts](#)

Contrôleur API - Expose les routes HTTP, applique les Guards et transmet les données au service métier. Routes principales : GET /api/v1/users/assignable, GET /api/v1/users, POST /api/v1/users, PATCH /api/v1/users/:id/status, PATCH /api/v1/users/:id/roles. Éléments déclarés : UsersController. Rôles cités : ADMIN, REFERENT_LABORATOIRE, RESPONSABLE_VEILLE.

Taille actuelle : 1 650 octets

[backend/src/users/users.module.ts](#)

Module NestJS - Assemble les contrôleurs, services, repositories et dépendances du module « users ». Éléments déclarés : UsersModule.

Taille actuelle : 667 octets

[backend/src/users/users.service.spec.ts](#)

Test automatisé - Vérifie le comportement du composant ou service associé avec Vitest.

Taille actuelle : 970 octets

[backend/src/users/users.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « users » et dialogue avec les repositories ou services associés. Éléments déclarés : UsersService. Rôles cités : ADMIN.

Taille actuelle : 5 035 octets

Backend / validation

[backend/src/validation/dto/review-watch-item.dto.ts](#)

DTO de validation - Décrit et valide les données acceptées par une route avant leur arrivée dans le service. Éléments déclarés : ReviewWatchItemDto.

Taille actuelle : 429 octets

[backend/src/validation/dto/workflow-comment.dto.ts](#)

DTO de validation - Décrit et valide les données acceptées par une route avant leur arrivée dans le service. Éléments déclarés : WorkflowCommentDto.

Taille actuelle : 172 octets

[backend/src/validation/entities/review.entity.ts](#)

Entité TypeORM - Mappe la table PostgreSQL « reviews », ses colonnes et ses relations. Éléments déclarés : Review.

Taille actuelle : 959 octets

[backend/src/validation/validation.controller.ts](#)

Contrôleur API - Expose les routes HTTP, applique les Guards et transmet les données au service métier. Routes principales : POST /api/v1/watch-items/:id/review, POST /api/v1/watch-items/:id/publish, POST /api/v1/watch-items/:id/archive, GET /api/v1/watch-items/:id/reviews. Éléments déclarés : ValidationController. Rôles cités : ADMIN, OPERATEUR_VEILLE, REFERENT_LABORATOIRE, RESPONSABLE_VEILLE.

Taille actuelle : 1 846 octets

[backend/src/validation/validation.module.ts](#)

Module NestJS - Assemble les contrôleurs, services, repositories et dépendances du module « validation ». Éléments déclarés : ValidationModule.

Taille actuelle : 804 octets

[backend/src/validation/validation.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « validation » et dialogue avec les repositories ou services associés. Éléments déclarés : ValidationService.

Taille actuelle : 6 201 octets

Backend / vitest.config.e2e.mts

[backend/vitest.config.e2e.mts](#)

Configuration des tests - Définit l'environnement, les chemins TypeScript et la sélection des tests Vitest.

Taille actuelle : 260 octets

Backend / vitest.config.e2e.ts

[backend/vitest.config.e2e.ts](#)

Configuration des tests - Définit l'environnement, les chemins TypeScript et la sélection des tests Vitest.

Taille actuelle : 256 octets

Backend / vitest.config.mts

[backend/vitest.config.mts](#)

Configuration des tests - Définit l'environnement, les chemins TypeScript et la sélection des tests Vitest.

Taille actuelle : 202 octets

Backend / watch-items

[backend/src/watch-items/constants/watch-status.constants.ts](#)

Constantes métier - Centralise des statuts ou valeurs autorisées pour éviter les chaînes dispersées. Éléments déclarés : WatchStatus, WATCH_STATUS.

Taille actuelle : 267 octets

[backend/src/watch-items/deduplication.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « watch-items » et dialogue avec les repositories ou services associés. Éléments déclarés : DeduplicationService.

Taille actuelle : 1 691 octets

[backend/src/watch-items/entities/watch-item-keyword.entity.ts](#)

Entité TypeORM - Mappe la table PostgreSQL « watch_item_keywords », ses colonnes et ses relations. Éléments déclarés : WatchItemKeyword.

Taille actuelle : 939 octets

[backend/src/watch-items/entities/watch-item-topic.entity.ts](#)

Entité TypeORM - Mappe la table PostgreSQL « watch_item_topics », ses colonnes et ses relations. Éléments déclarés : WatchItemTopic.

Taille actuelle : 917 octets

[backend/src/watch-items/entities/watch-item.entity.ts](#)

Entité TypeORM - Mappe la table PostgreSQL « watch_items », ses colonnes et ses relations. Éléments déclarés : WatchItem.

Taille actuelle : 3 376 octets

[backend/src/watch-items/entities/watch-version.entity.ts](#)

Entité TypeORM - Mappe la table PostgreSQL « watch_versions », ses colonnes et ses relations. Éléments déclarés : WatchVersion.

Taille actuelle : 751 octets

[backend/src/watch-items/interfaces/normalized-watch-item.interface.ts](#)

Contrat TypeScript - Définit la forme des données échangées sans produire de code à l'exécution. Éléments déclarés : NormalizedWatchItem.

Taille actuelle : 385 octets

[backend/src/watch-items/watch-items.controller.ts](#)

Contrôleur API - Expose les routes HTTP, applique les Guards et transmet les données au service métier. Routes principales : GET /api/v1/watch-items, GET /api/v1/watch-items/:id. Éléments déclarés : WatchItemsController. Rôles cités : ADMIN, LECTEUR, OPERATEUR_VEILLE, REFERENT_LABORATOIRE, RESPONSABLE_VEILLE.

Taille actuelle : 1 017 octets

[backend/src/watch-items/watch-items.module.ts](#)

Module NestJS - Assemble les contrôleurs, services, repositories et dépendances du module « watch-items ». Éléments déclarés : WatchItemsModule.

Taille actuelle : 788 octets

[backend/src/watch-items/watch-items.service.ts](#)

Service métier NestJS - Implémente les règles du domaine « watch-items » et dialogue avec les repositories ou services associés. Éléments déclarés : WatchItemsService, IngestionResult. Rôles cités : ADMIN, OPERATEUR_VEILLE, REFERENT_LABORATOIRE, RESPONSABLE_VEILLE.

Taille actuelle : 4 330 octets

Frontend / .gitignore

[frontend/.gitignore](#)

Fichier projet - Participe au fonctionnement général du projet.

Taille actuelle : 253 octets

Frontend / App.vue

[frontend/src/App.vue](#)

Composant Vue - Composant d'interface réutilisable ou structure globale de l'application.

Taille actuelle : 1 200 octets

Frontend / assets

[frontend/src/assets/logos/logo-veille-microscope.png](#)

Ressource graphique - Image utilisée pour l'identité visuelle ou l'icône de l'application.

Taille actuelle : 160 653 octets

Frontend / i18n

[frontend/src/i18n/errors.spec.ts](#)

Test automatisé - Vérifie le comportement du composant ou service associé avec Vitest.

Taille actuelle : 441 octets

[frontend/src/i18n/errors.ts](#)

Traduction - Centralise les libellés français ou la configuration française de l'interface. Éléments déclarés : errorFr.

Taille actuelle : 2 175 octets

[frontend/src/i18n/labels.spec.ts](#)

Test automatisé - Vérifie le comportement du composant ou service associé avec Vitest.

Taille actuelle : 597 octets

[frontend/src/i18n/labels.ts](#)

Traduction - Centralise les libellés français ou la configuration française de l'interface. Éléments déclarés : labelFr, optionsFr.

Taille actuelle : 2 164 octets

[frontend/src/i18n/primevue-fr.ts](#)

Traduction - Centralise les libellés français ou la configuration française de l'interface. Éléments déclarés : primevueFr.

Taille actuelle : 4 934 octets

Frontend / index.html

[frontend/index.html](#)

Entrée HTML - Document HTML racine dans lequel Vue monte l'application.

Taille actuelle : 370 octets

Frontend / layouts

[frontend/src/layouts/AppLayout.vue](#)

Composant Vue - Composant d'interface réutilisable ou structure globale de l'application. Rôles cités : ADMIN, LECTEUR, OPERATEUR_VEILLE, REFERENT_LABORATOIRE, RESPONSABLE_VEILLE.

Taille actuelle : 11 939 octets

Frontend / main.ts

[frontend/src/main.ts](#)

Fichier projet - Participe au fonctionnement général du projet.

Taille actuelle : 610 octets

Frontend / package-lock.json

[frontend/package-lock.json](#)

Fichier généré npm - Fige les versions exactes et l'arbre des dépendances pour des installations reproductibles.

Taille actuelle : 102 298 octets

Frontend / package.json

[frontend/package.json](#)

Configuration npm - Déclare les scripts, dépendances et métadonnées du paquet concerné.

Taille actuelle : 767 octets

Frontend / public

[frontend/public/favicon.svg](#)

Ressource graphique - Image utilisée pour l'identité visuelle ou l'icône de l'application.

Taille actuelle : 9 522 octets

Frontend / router

[frontend/src/router/index.ts](#)

Routeur frontend - Déclare les pages, le chargement différé et les contrôles d'accès selon la session et les rôles.

Taille actuelle : 4 750 octets

Frontend / services

[frontend/src/services/actions.service.ts](#)

Service API frontend - Centralise les appels Axios vers le backend et les types de données utilisés par les pages. Éléments déclarés : FollowUpAction, getActions, getAllActions, createAction, updateAction.

Taille actuelle : 1 103 octets

[frontend/src/services/api.ts](#)

Service API frontend - Centralise les appels Axios vers le backend et les types de données utilisés par les pages.

Taille actuelle : 896 octets

[frontend/src/services/audit.service.ts](#)

Service API frontend - Centralise les appels Axios vers le backend et les types de données utilisés par les pages. Éléments déclarés : getAuditLogs.

Taille actuelle : 89 octets

[frontend/src/services/connector-config.ts](#)

Service API frontend - Centralise les appels Axios vers le backend et les types de données utilisés par les pages. Éléments déclarés : buildConnectorConfig.

Taille actuelle : 972 octets

[frontend/src/services/dashboard.service.ts](#)

Service API frontend - Centralise les appels Axios vers le backend et les types de données utilisés par les pages. Éléments déclarés : DashboardStats, getDashboard.

Taille actuelle : 163 octets

[frontend/src/services/favorites.service.ts](#)

Service API frontend - Centralise les appels Axios vers le backend et les types de données utilisés par les pages. Éléments déclarés : getFavorites, addFavorite, removeFavorite.

Taille actuelle : 237 octets

[frontend/src/services/health.service.ts](#)

Service API frontend - Centralise les appels Axios vers le backend et les types de données utilisés par les pages. Éléments déclarés : getHealth, retryConnector.

Taille actuelle : 181 octets

[frontend/src/services/notifications.service.ts](#)

Service API frontend - Centralise les appels Axios vers le backend et les types de données utilisés par les pages. Éléments déclarés : getNotifications, markNotificationRead.

Taille actuelle : 188 octets

[frontend/src/services/qualification.service.ts](#)

Service API frontend - Centralise les appels Axios vers le backend et les types de données utilisés par les pages. Éléments déclarés : Qualification, QualificationPayload, getQualification, qualifyWatchItem.

Taille actuelle : 857 octets

[frontend/src/services/reports.service.ts](#)

Service API frontend - Centralise les appels Axios vers le backend et les types de données utilisés par les pages. Éléments déclarés : getReports, generateReport, downloadReport.

Taille actuelle : 289 octets

[frontend/src/services/saved-views.service.ts](#)

Service API frontend - Centralise les appels Axios vers le backend et les types de données utilisés par les pages. Éléments déclarés : getSavedViews, createSavedView, deleteSavedView.

Taille actuelle : 309 octets

[frontend/src/services/search.service.ts](#)

Service API frontend - Centralise les appels Axios vers le backend et les types de données utilisés par les pages. Éléments déclarés : SearchParams, searchWatchItems.

Taille actuelle : 471 octets

[frontend/src/services/sources.service.ts](#)

Service API frontend - Centralise les appels Axios vers le backend et les types de données utilisés par les pages. Éléments déclarés : Source, CreateSourcePayload, CollectionRunResult, getSources, createSource, createConnector, updateSourceStatus, testConnector, runConnector.

Taille actuelle : 1 586 octets

[frontend/src/services/subscriptions.service.ts](#)

Service API frontend - Centralise les appels Axios vers le backend et les types de données utilisés par les pages. Éléments déclarés : getSubscriptions, getSubscriptionOptions, createSubscription, deleteSubscription.

Taille actuelle : 337 octets

[frontend/src/services/taxonomy.service.ts](#)

Service API frontend - Centralise les appels Axios vers le backend et les types de données utilisés par les pages.

Taille actuelle : 45 octets

[frontend/src/services/taxonomy/taxonomy.service.ts](#)

Service API frontend - Centralise les appels Axios vers le backend et les types de données utilisés par les pages. Éléments déclarés : Topic, Domain, Laboratory, KeywordSynonym, Keyword, getTopics, createTopic, updateTopic, getDomains, createDomain.

Taille actuelle : 2 493 octets

[frontend/src/services/users.service.ts](#)

Service API frontend - Centralise les appels Axios vers le backend et les types de données utilisés par les pages. Éléments déclarés : AssignableUser, getUsers.

Taille actuelle : 223 octets

[frontend/src/services/validation.service.ts](#)

Service API frontend - Centralise les appels Axios vers le backend et les types de données utilisés par les pages. Éléments déclarés : Review, reviewWatchItem, publishWatchItem, archiveWatchItem, getReviews.

Taille actuelle : 927 octets

[frontend/src/services/watch-items.service.ts](#)

Service API frontend - Centralise les appels Axios vers le backend et les types de données utilisés par les pages. Éléments déclarés : WatchItem, getWatchItems, getWatchItem.

Taille actuelle : 575 octets

Frontend / stores

[frontend/src/stores/auth.ts](#)

Store Pinia - Conserve l'état partagé de l'application et expose des actions réactives. Éléments déclarés : currentUser, useAuthStore.

Taille actuelle : 1 443 octets

Frontend / style.css

[frontend/src/style.css](#)

Styles globaux - Définit Tailwind, les couleurs, dimensions et adaptations visuelles communes.

Taille actuelle : 1 211 octets

Frontend / tsconfig.app.json

[frontend/tsconfig.app.json](#)

Configuration TypeScript - Définit les règles de compilation TypeScript pour ce périmètre.

Taille actuelle : 430 octets

Frontend / tsconfig.json

[frontend/tsconfig.json](#)

Configuration TypeScript - Définit les règles de compilation TypeScript pour ce périmètre.

Taille actuelle : 119 octets

Frontend / tsconfig.node.json

[frontend/tsconfig.node.json](#)

Configuration TypeScript - Définit les règles de compilation TypeScript pour ce périmètre.

Taille actuelle : 558 octets

Frontend / views

[frontend/src/views/ActionsView.vue](#)

Page Vue - Interface utilisateur de la page « Actions » : charge les données, gère les actions et affiche les composants PrimeVue. Rôles cités : ADMIN, REFERENT_LABORATOIRE, RESPONSABLE_VEILLE.

Taille actuelle : 3 077 octets

[frontend/src/views/AuditLogsView.vue](#)

Page Vue - Interface utilisateur de la page « AuditLogs » : charge les données, gère les actions et affiche les composants PrimeVue.

Taille actuelle : 2 570 octets

[frontend/src/views/DashboardView.vue](#)

Page Vue - Interface utilisateur de la page « Dashboard » : charge les données, gère les actions et affiche les composants PrimeVue. Rôles cités : ADMIN, OPERATEUR_VEILLE, REFERENT_LABORATOIRE, RESPONSABLE_VEILLE.

Taille actuelle : 3 058 octets

[frontend/src/views/LoginView.vue](#)

Page Vue - Interface utilisateur de la page « Login » : charge les données, gère les actions et affiche les composants PrimeVue.

Taille actuelle : 5 558 octets

[frontend/src/views/NotificationsView.vue](#)

Page Vue - Interface utilisateur de la page « Notifications » : charge les données, gère les actions et affiche les composants PrimeVue.

Taille actuelle : 4 526 octets

[frontend/src/views/ReportsView.vue](#)

Page Vue - Interface utilisateur de la page « Reports » : charge les données, gère les actions et affiche les composants PrimeVue.

Taille actuelle : 9 501 octets

[frontend/src/views/SourcesView.vue](#)

Page Vue - Interface utilisateur de la page « Sources » : charge les données, gère les actions et affiche les composants PrimeVue. Rôles cités : ADMIN, OPERATEUR_VEILLE, RESPONSABLE_VEILLE.

Taille actuelle : 21 567 octets

[frontend/src/views/SubscriptionsView.vue](#)

Page Vue - Interface utilisateur de la page « Subscriptions » : charge les données, gère les actions et affiche les composants PrimeVue.

Taille actuelle : 5 477 octets

[frontend/src/views/SystemHealthView.vue](#)

Page Vue - Interface utilisateur de la page « SystemHealth » : charge les données, gère les actions et affiche les composants PrimeVue.

Taille actuelle : 2 297 octets

[frontend/src/views/TaxonomyView.vue](#)

Page Vue - Interface utilisateur de la page « Taxonomy » : charge les données, gère les actions et affiche les composants PrimeVue.

Taille actuelle : 16 960 octets

[frontend/src/views/UsersView.vue](#)

Page Vue - Interface utilisateur de la page « Users » : charge les données, gère les actions et affiche les composants PrimeVue. Rôles cités : ADMIN.

Taille actuelle : 10 828 octets

[frontend/src/views/WatchItemDetailView.vue](#)

Page Vue - Interface utilisateur de la page « WatchItemDetail » : charge les données, gère les actions et affiche les composants PrimeVue. Rôles cités : ADMIN, OPERATEUR_VEILLE, REFERENT_LABORATOIRE, RESPONSABLE_VEILLE.

Taille actuelle : 15 589 octets

[frontend/src/views/WatchItemQualificationView.vue](#)

Page Vue - Interface utilisateur de la page « WatchItemQualification » : charge les données, gère les actions et affiche les composants PrimeVue.

Taille actuelle : 13 855 octets

[frontend/src/views/WatchItemsView.vue](#)

Page Vue - Interface utilisateur de la page « WatchItems » : charge les données, gère les actions et affiche les composants PrimeVue. Rôles cités : ADMIN, OPERATEUR_VEILLE, REFERENT_LABORATOIRE, RESPONSABLE_VEILLE.

Taille actuelle : 13 659 octets

Frontend / vite.config.ts

[frontend/vite.config.ts](#)

Configuration Vite - Configure le serveur de développement, les plugins Vue/Tailwind et la construction frontend.

Taille actuelle : 203 octets

Frontend / vitest.config.ts

[frontend/vitest.config.ts](#)

Configuration des tests - Définit l'environnement, les chemins TypeScript et la sélection des tests Vitest.

Taille actuelle : 175 octets

Racine du projet

[.gitignore](#)

Fichier projet - Participe au fonctionnement général du projet.

Taille actuelle : 370 octets

[package-lock.json](#)

Fichier généré npm - Fige les versions exactes et l'arbre des dépendances pour des installations reproductibles.

Taille actuelle : 11 905 octets

[package.json](#)

Configuration npm - Déclare les scripts, dépendances et métadonnées du paquet concerné.

Taille actuelle : 99 octets

[README.md](#)

Documentation - Guide d'installation, de démarrage, d'utilisation et de dépannage du dépôt.

Taille actuelle : 14 188 octets

5. Explication des mécanismes de code essentiels

5.1 main.ts : sécuriser toutes les requêtes

Le point d'entrée installe des protections transversales avant d'écouter le port. ValidationPipe supprime/refuse les propriétés inconnues et transforme les paramètres. Helmet ajoute des en-têtes de sécurité. CORS n'accepte que FRONTEND_URL. Les corps sont limités à 1 Mo.

```
app.use(helmet());
app.use(cookieParser());
app.use(json({ limit: '1mb' }));
app.useGlobalPipes(new ValidationPipe({
  whitelist: true,
  forbidNonWhitelisted: true,
  transform: true,
}));
```

5.2 AppModule : composition de l'application

AppModule enregistre explicitement toutes les entités TypeORM, garde synchronize=false, active le scheduler, l'EventEmitter et le throttling, puis importe chaque module métier. Une entité oubliée ici ne sera pas connue de la connexion utilisée par NestJS.

5.3 Contrôleur → service → repository

Le contrôleur traduit HTTP en appel métier. Le service contrôle les règles et utilise un repository TypeORM. Le repository transforme l'opération en SQL. Cette chaîne évite de mélanger validation, autorisation, métier et base de données.

```
@Patch('/:id/status')
@Roles('ADMIN')
setStatus(@Param('id', ParseIntPipe) id: number,
  @Body() dto: UpdateSourceStatusDto) {
  return this.sourcesService.setActive(id, dto.active);
}
```

5.4 Guards et rôles

JwtAuthGuard demande une identité valide. RolesGuard lit les rôles déposés par @Roles et vérifie ceux de request.user. Le routeur Vue applique la même logique pour l'affichage, mais le backend reste l'autorité de sécurité.

5.5 Transactions

Les opérations qui modifient plusieurs tables utilisent une transaction. Une qualification, une publication ou une collecte ne doit pas laisser une moitié de résultat en base. En cas d'erreur, rollback annule l'ensemble.

5.6 Événements

ValidationService publie un événement après la validation de la transaction. NotificationsService traite l'événement sans coupler directement la logique de validation aux abonnements et notifications.

5.7 Recherche plein texte

SearchService utilise to_tsvector et plainto_tsquery. L'index GIN de migration accélère les recherches sur titre et résumé. Les paramètres TypeORM empêchent la concaténation directe de la saisie utilisateur dans le SQL.

5.8 Déduplication

La stratégie utilise les identifiants les plus fiables en premier. Elle conserve une version lorsqu'un élément connu a changé et retourne un statut métier permettant à CollectionService d'incrémenter les bons compteurs.

5.9 Store d'authentification

Le store Pinia garde seulement l'utilisateur en mémoire. Au rechargement, /auth/me vérifie le cookie HttpOnly. Axios utilise withCredentials=true. Une réponse 401 déclenche auth:unauthorized et vide la session.

5.10 Chargement différé des pages

Les vues sont importées avec () => import(...). Vite crée des fichiers JavaScript séparés et le navigateur ne charge une page que lorsqu'elle est visitée, ce qui réduit le poids initial.

6. Ordre conseillé pour comprendre le projet

Ordre	Fichiers	Question répondue
1	backend/src/main.ts, app.module.ts	Comment l'API démarre-t-elle ?
2	auth/*, users/*, roles/*	Comment l'accès est-il contrôlé ?
3	sources/*, connectors/*	D'où viennent les données ?
4	collection/*, watch-items/*	Comment deviennent-elles des veilles ?
5	taxonomy/*, qualification/*	Comment sont-elles enrichies ?
6	validation/*, actions/*	Comment sont-elles décidées et suivies ?
7	search/*, favorites/*, saved-views/*	Comment les retrouver ?
8	subscriptions/*, notifications/*, reports/*	Comment les diffuser ?
9	dashboard/*, audit/*, health/*	Comment piloter la plateforme ?
10	frontend/src/router, stores, services, views	Comment l'interface utilise-t-elle l'API ?

Cette lecture suit le même trajet que l'information : entrée sécurisée, acquisition, traitement, décision, diffusion et pilotage.